



THE GEORGE
WASHINGTON
UNIVERSITY

WASHINGTON, DC

DISTRIBUTED SYSTEMS CS6421

INTRO TO DISTRIBUTED SYSTEMS AND THE CLOUD COMPUTING

Prof. Roozbeh Haghazadeh

Slides Credit:

Prof. Tim Wood and Prof. Roozbeh Haghazadeh

PROF. ROOZBEH HAGHNAZAR

- Started Programming in 1991 with Commodore 64
- Played several roles in technology, such as Developer, Modeler, Designer, Architect, Leader, CTO, etc.
- Teach Software Eng., Distributed Systems, Data Base Design Principles, Data Visualization, Operating System.
- Data Science Lead in NWITS-USGS
- Tech Lead in Spirent Communications





ABOUT THIS COURSE

- Be prepared! (course prerequisites)
 - CSCI 6212 Algorithms (or undergrad algorithms course)
 - An undergraduate operating systems course
- Be involved!
 - “Raise hand”, ask questions , discuss, etc.
 - Asynchronous opportunities will be available
- Be ready to code!
 - You will need to use **Go, Python** for your assignments
 - Mostly group projects



CLASSES

- 2.5 hours is a long time for lectures!
 - We will try to break it up – discussions, demos, live coding
 - Some lectures may end early, with additional asynchronous material
- We want to make the best course we can for you!



RESOURCES

- Slack: (linked from website, join after class)
- GitHub for collecting assignments
- Blackboard for grades, class meetings, and office hours
- Visual Studio Code – recommended IDE
 - Live share plugin allows group collaboration / help in office hours
- Repl.it – simple online editor for quick programming exercises
 - You can login with GitHub credentials if you want to save copies

SEMESTER OUTLINE

- Building Blocks
 - Introduction to Distributed System and Cloud
 - Scalable Execution: Processes, threads, VMs, containers, parallelism vs concurrency
 - Communication: RPC, Message Oriented, Stream Oriented
- **Principles** of Distributed Systems
 - Coordination: Synchronization, Consistency, and Consensus
 - Reliability: Replication and Fault Tolerance
 - Performance: Metrics and Modeling Large Scale Systems
- Distributed Systems in **Practice**
 - Grid Computing
 - Cloud Computing
 - Web, Mobile, and IoT

4 programming assignments
Midterm ???
Large group project

History of Computers

Timeline and Ordering Activities

INTRODUCTION

- Computer systems are undergoing revolution.
- Two advances in technology changed the game
 - 8bit -> 16bit -> 32bit -> 64bit microprocessors
 - From a machine that cost \$10M and executed 1 inst./sec we have come to machine that cost \$1000 and execute 1 billion inst./sec
 - Computer networks LAN/WAN
 - From 64 Kbit/sec to Gigabit/Sec





INTRODUCTION

- If the automotive industry had advanced at the same rapid pace as computer science, today we could purchase a Rolls-Royce for just one dollar and get a billion miles per gallon





WHAT IS THE CLOUD?

WHAT IS THE CLOUD

- Giant warehouses
- 10s of thousands of servers
- Petabytes of storage
- 10s of thousands of Processor cores
-Interconnected....



WHY INFRASTRUCTURE?

- Why do we need this amount of infrastructures?
 - Encyclopedia Britannica
 - - 40,000+ articles
 - - 32 hard bound volumes (32,640 pages)
 - Wikipedia
 - - 5,512,202 articles (in English)
 - - More than **5 TB** of text (about 7,500 CDs)
 - - More than 2000 volumes



AND THEN BIG DATA

- Why do we need this amount of infrastructures?
 - Airbus A350
 - Contains around 6000 sensors across the entire plane that generates 2.5TB Data per day
 - Airbus A380-100
 - Expected to take the skies in 2020
 - Contains 10000 sensors just in each wings
 - Facebook
 - 20 TB photos each week
 - Google
 - 20000TB Data processing per day in 2008



AND THEN BIG DATA

- Google Search Statistics

The average figure of how many people use Google a day, which translates into at least 2 trillion searches per year, 3.8 million searches per minute, 228 million searches per hour, and 5.6 billion searches per day.

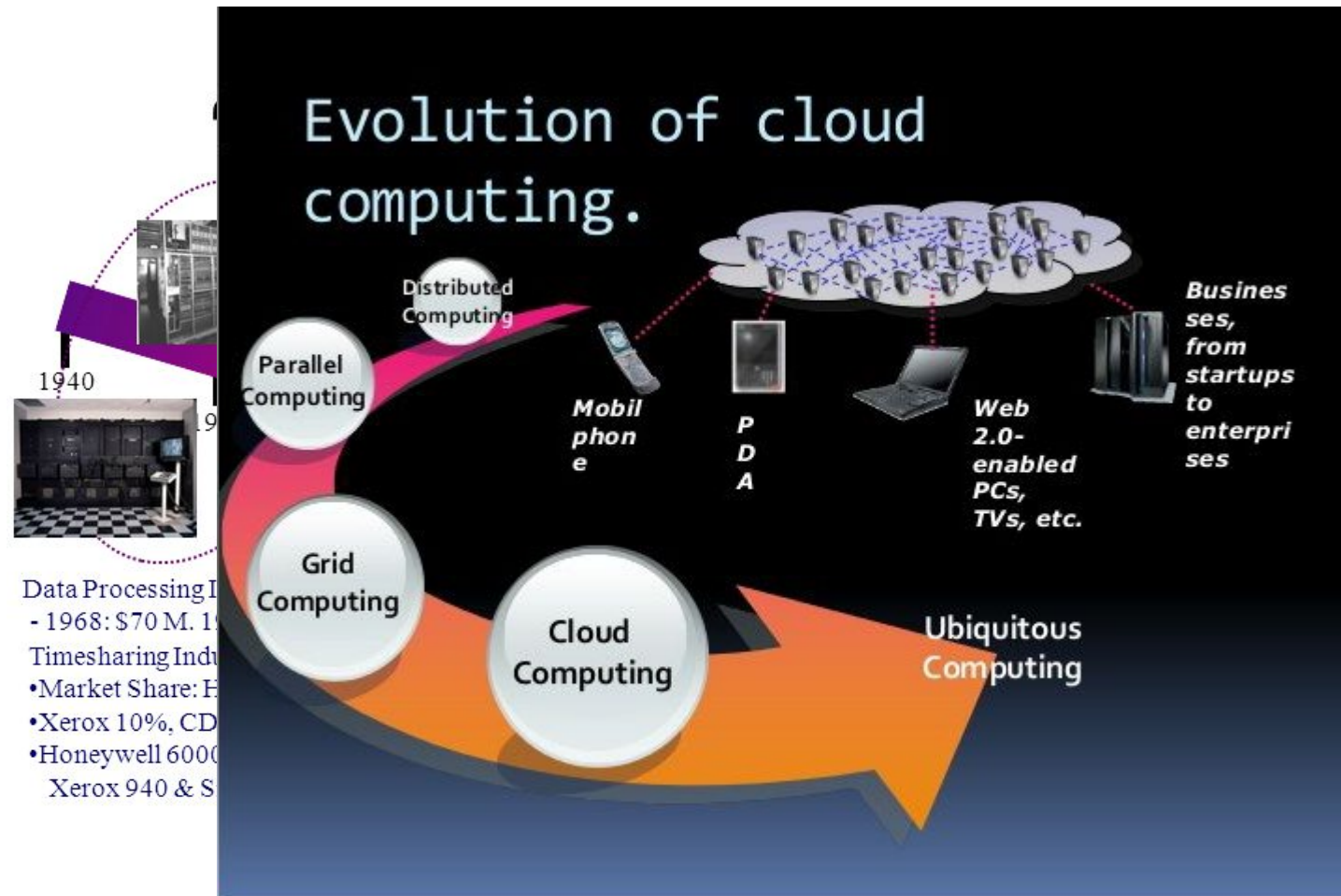
- How much data do we generate?

According to the Forbes statistics:

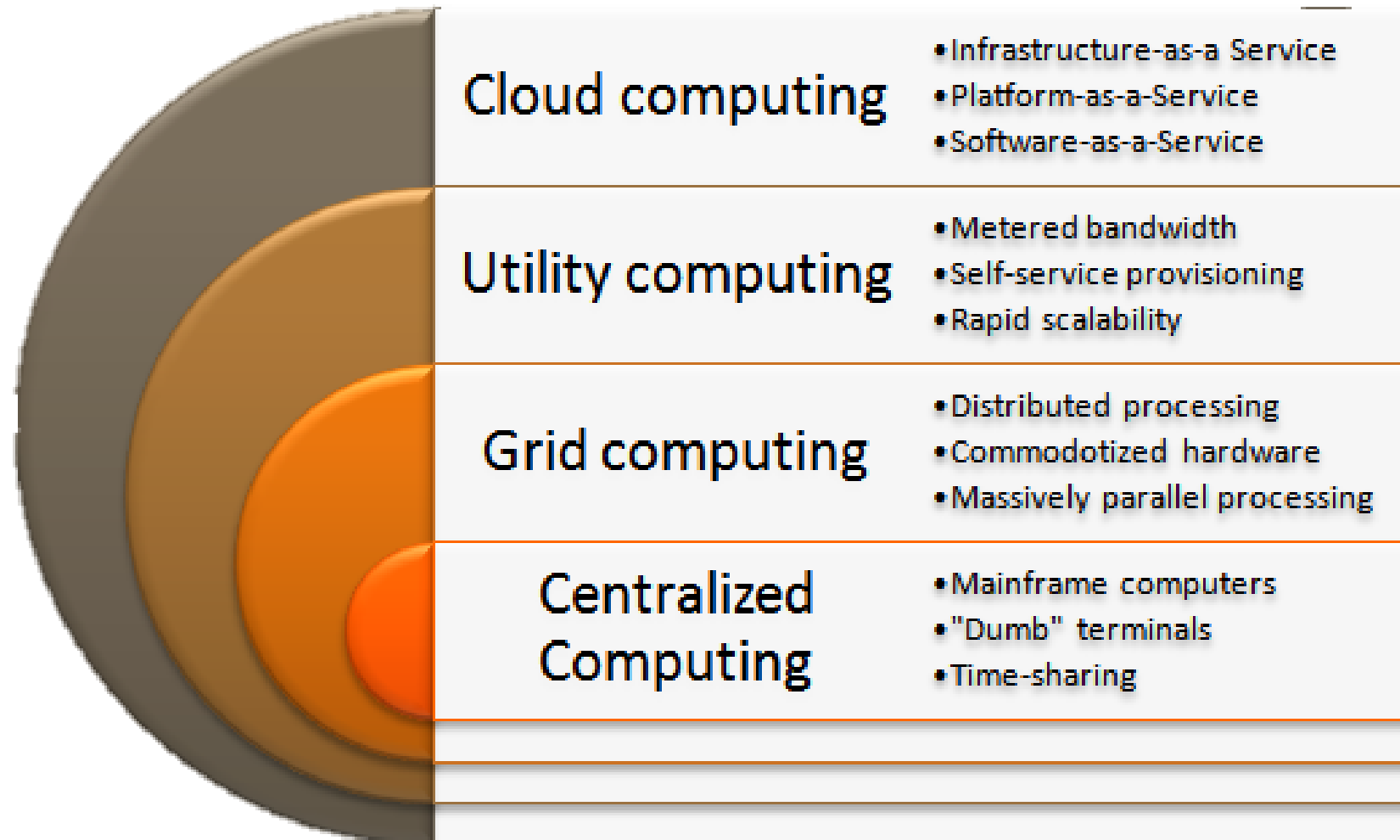
- 2.5 quintillion bytes of data created each day
- Over the last two years alone 90 percent of the data in the world was generated.

KB	Kilo Byte	1 thousand bytes
MB	Mega Byte	1 million bytes
GB	Giga Byte	1 billion bytes
TB	Tera Byte	1 trillion bytes
PB	Peta Byte	1 quadrillion bytes
EB	Exa Byte	1 quintillion bytes

HISTORY OF CLOUD COMPUTING



HISTORY OF CLOUD COMPUTING





WHAT'S NEW

- There are four new features in the new generation of distributed and cloud systems:
 - Massive Scale
 - On-Demand Access: Pay-as-you-go
 - Data Intensive Nature: MBs became PBs and XBs
 - New Cloud Programming Paradigms: Map/Reduce Hadoop, Unstructured Data

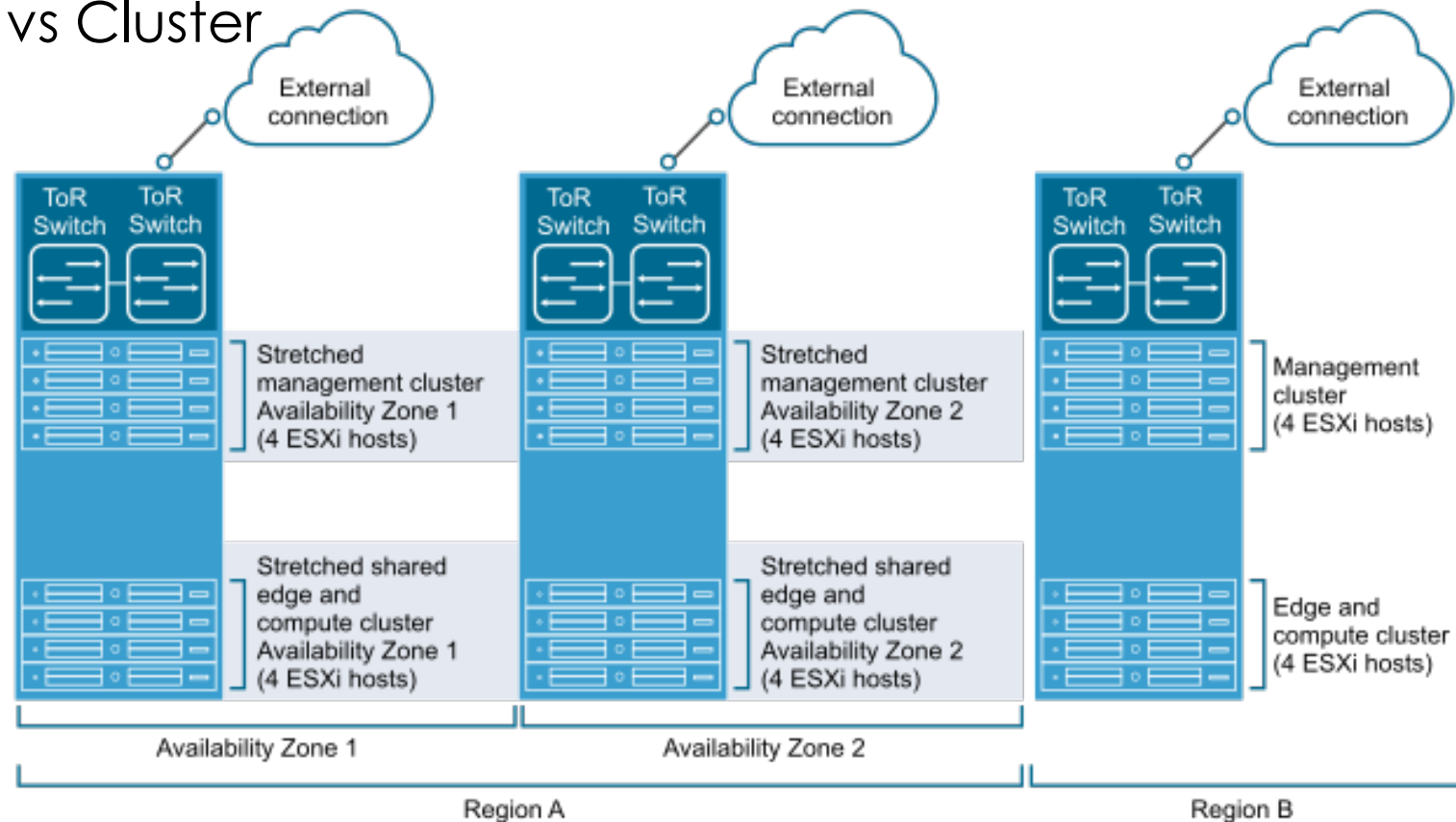
*_{AA}S CLASSIFICATION

- HaaS : Hardware as a Service
Hardware and backbone
- IaaS: Infrastructure as a Service
AWS, Azure, GCP
- PaaS: Platform as a Service
Google App engine, AWS Elastic Beanstalk
- SaaS: Software as a Service
Google Doc, Dropbox

CLOUD IS A ...

- Cloud vs Distributed System vs Cluster

- Client Server Architecture





CLOUD IS A ...

- Can we say “ Cloud is a fancy word for a Distributed System?”



WHAT IS A DISTRIBUTED SYSTEM

- A distributed system is a collection of independent computers that appears to its users as a single coherent system. [Andrew Tanenbaum]
 - distributed system consists of components that are autonomous
 - users (be they people or programs) think they are dealing with a single system.
(Transparency)
 - distributed systems should also be relatively easy to expand or scale.
 - Heterogeneity
 - Concurrency



GOALS OF DS

- Making resources accessible
- Distribution Transparency
 - Access
 - Location
 - Migration
 - Relocation
 - Replication
 - Concurrency
 - Failure
- Openness
- Scalability



ACCESSIBILITY

- The main goal of a distributed system is to make it easy for the users and applications to access remote resources and to share them in a controlled and efficient way



TRANSPARENCY

- **Transparency** in simple words is defined as the concealment from the user and the application programmer of the separation of components in a **distributed system**, so that the **system** is perceived as a whole rather than as a collection of independent components.



OPENNESS

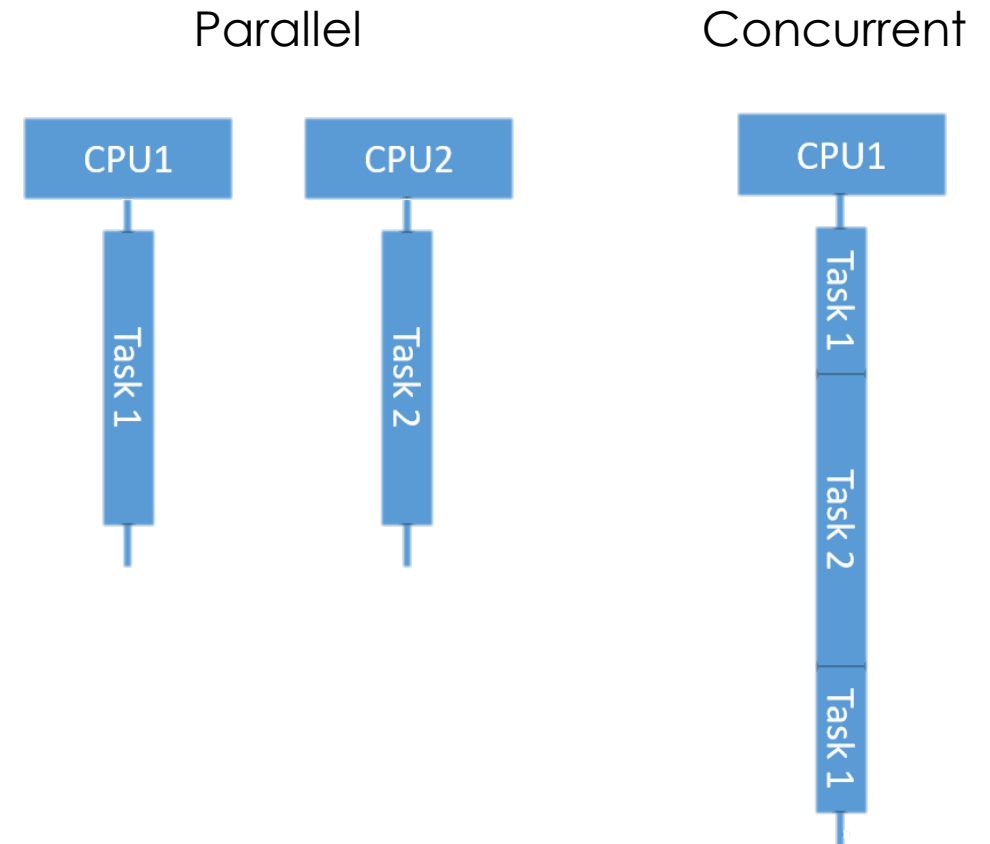
- An open distributed system is a system that offers services according to standard rules that describe the syntax and semantics of those services.

SCALABILITY

- **Scalability** means you can increase or reduce the capacity, diversity, power or abilities of your system. It can be measured along at least three different dimensions:
 - A system can be scalable with respect to its size (add more users/resources to the system – can be consider as Scale up)
 - A geographically scalable system is one in which the users may lie far apart (Scale out)
 - A system can be administratively scalable. It means that it can still be easy to manage even if it spans many independent administrative organizations.

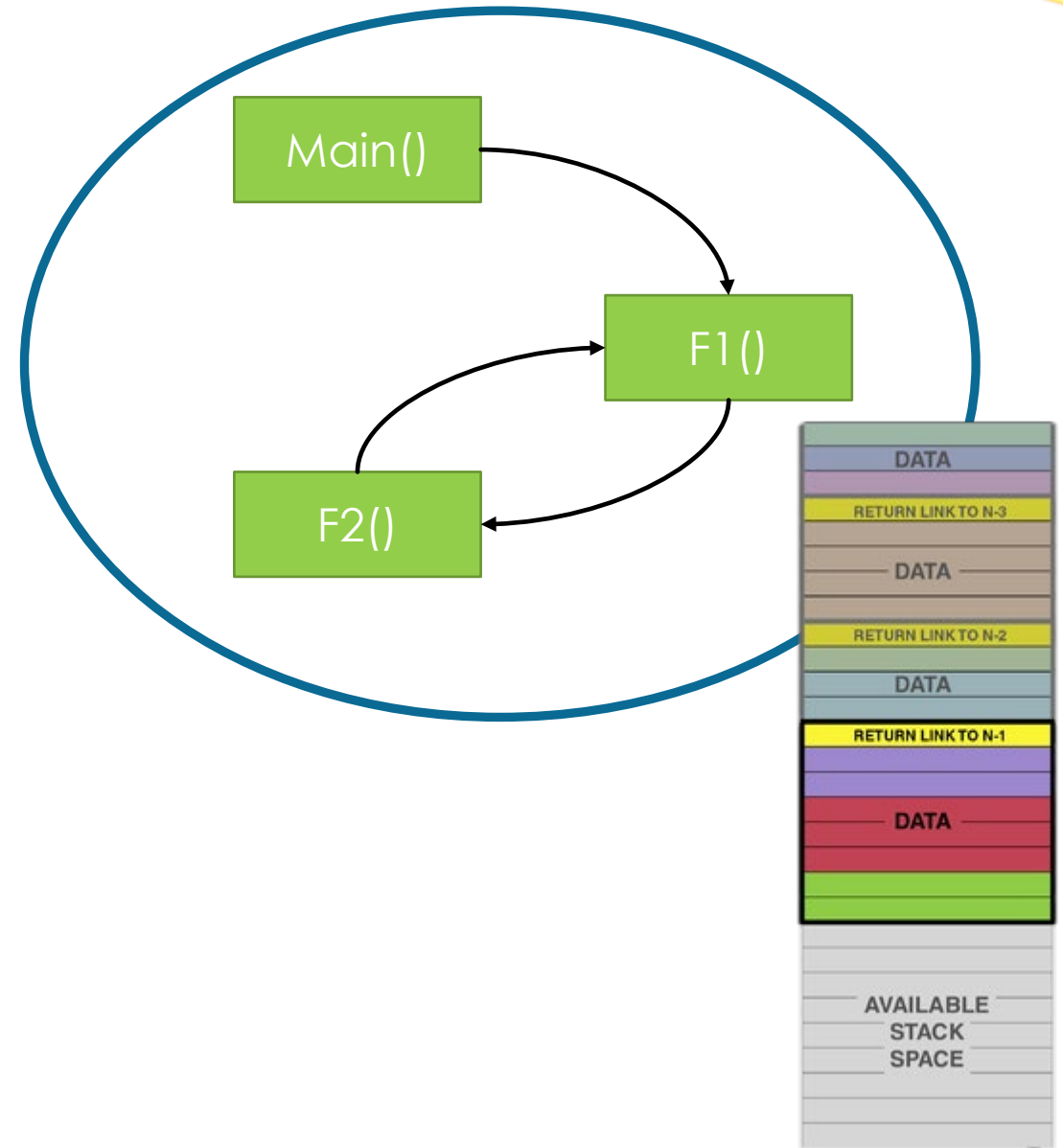
CONCURRENCY VS PARALLELISM

- Concurrency considers the checkpoints
- Parallelism considers time of progresses



PROCESS

- Process
- Stack
- Program Counter
- Heap
- Etc.



DISTRIBUTED

- Distributed System = Many Processes ?????

P1

P2

P3

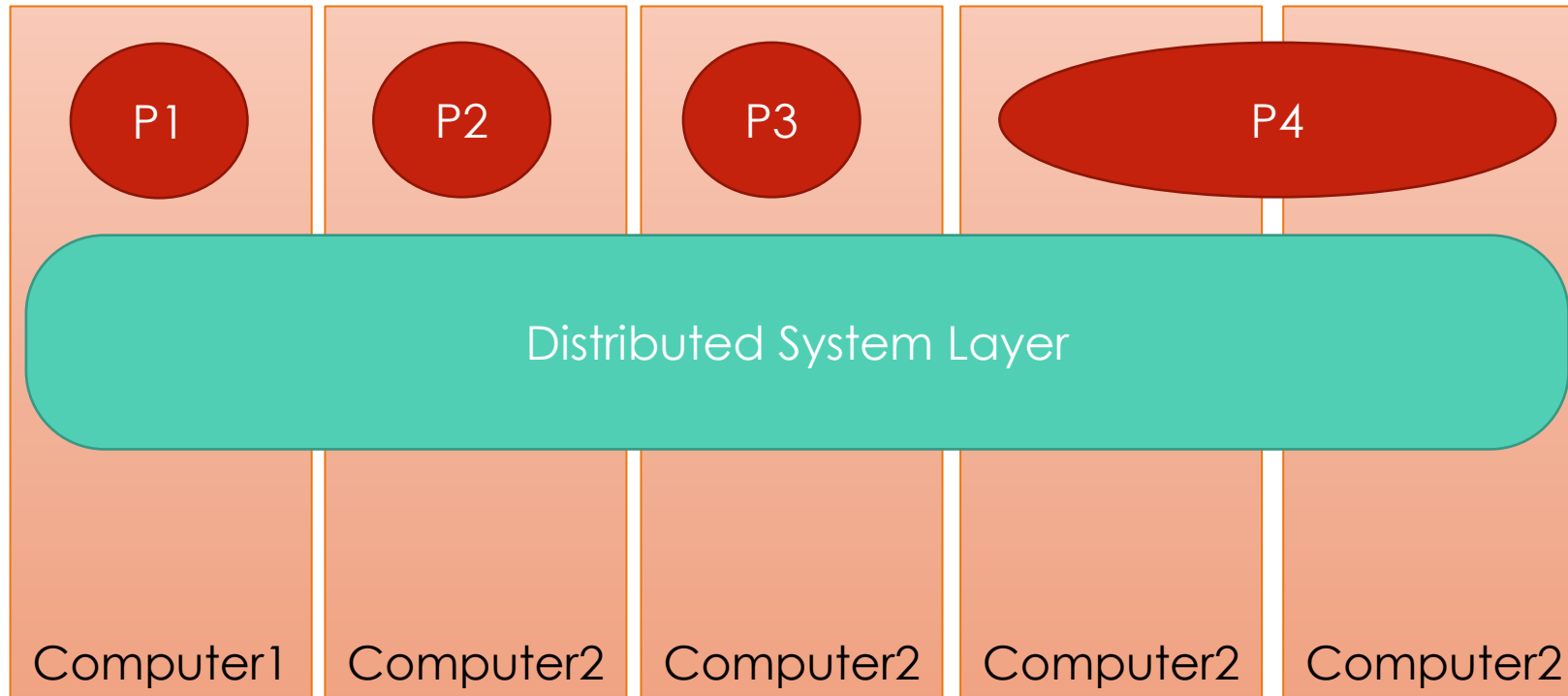
P4

..... Pn

Reliable or Unreliable Communication

HOW CAN WE HANDLE?

- Faster Computer Or Add Another Computer?



The background features abstract, flowing waves in shades of red and yellow, creating a dynamic and energetic feel. The waves are layered, with some appearing more prominent than others, and they curve across the frame.

HW 1: GO PARALLEL SUM

PARALLEL SUM

- Assignment Goals:
 - Learn the basics of the Go programming language
 - Familiarize yourself with the editing environment and Git
 - Build two types of distributed systems
- This is an **individual** assignment
 - You must write all your own code
 - You may discuss general ideas with other students and link them help documentation
 - You may give general advice for debugging and design, but you should **never** have your code open while looking at someone else's code!
 - This is more lenient than many classes, don't abuse it!

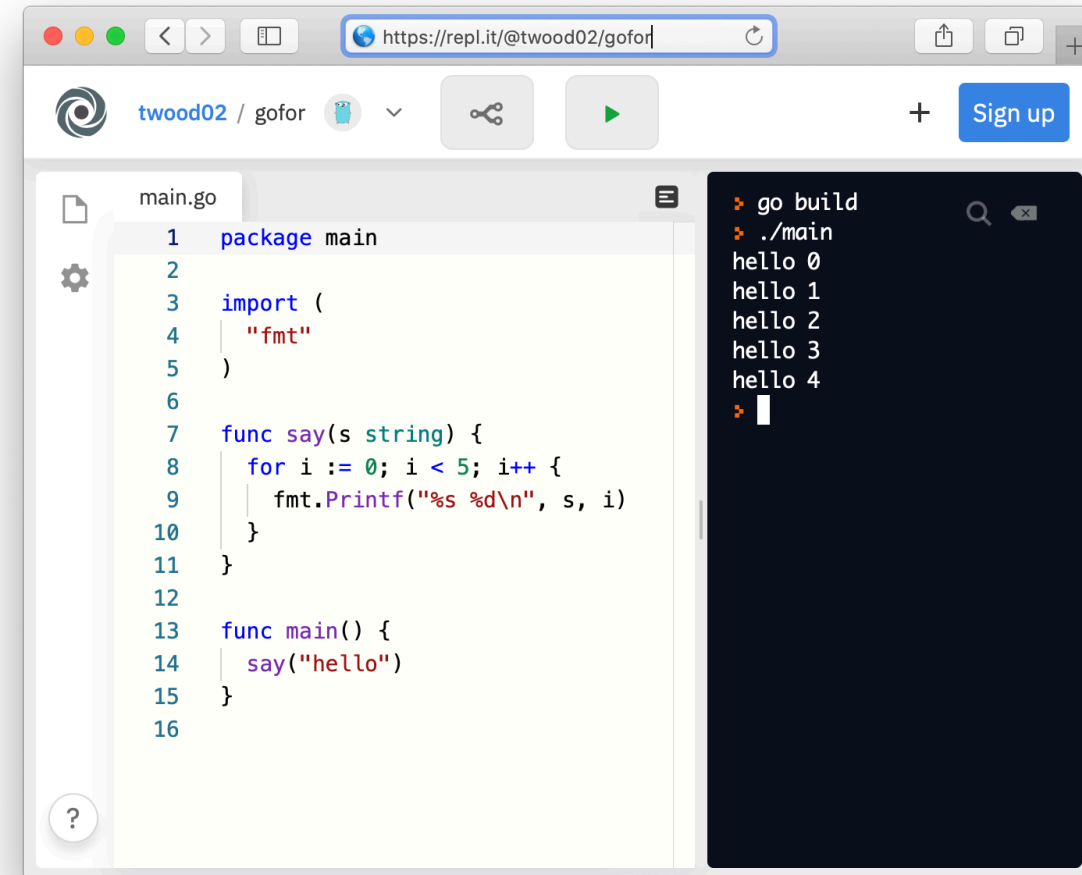
WHY GO?

- Go has become a very popular language for building distributed systems
- Born at Google by Robert Griesemer, Rob Pike and Ken Thompson (C/Unix)
- Power and performance of C, but with the convenience and safety of more modern languages
- Learn more: <https://golang.org/doc/faq>

“Go ... [attempted] to combine the ease of programming of an interpreted, dynamically typed language with the efficiency and safety of a statically typed, compiled language. It also aimed to be modern, with support for networked and multicore computing.”

PHASE 1: SEQUENTIAL SUM

- Starter code:
 - Reads a file and puts numbers in an array
- Your code:
 - Use a for loop and add up the numbers
 - Add command line parameter support
 - (this should be easy even if you've never touched go)
- Hint: Take a tour of Go
 - <https://tour.golang.org/list>



The screenshot shows a web-based Go Playground interface. The browser address bar displays `https://repl.it/@twood02/gofor`. The interface includes a user profile section for 'twood02' with a 'gofor' label, a share icon, a run button (green play icon), and a 'Sign up' button. The main area is divided into two panes. The left pane, titled 'main.go', contains the following Go code:

```
1 package main
2
3 import (
4     "fmt"
5 )
6
7 func say(s string) {
8     for i := 0; i < 5; i++ {
9         fmt.Printf("%s %d\n", s, i)
10    }
11 }
12
13 func main() {
14     say("hello")
15 }
16
```

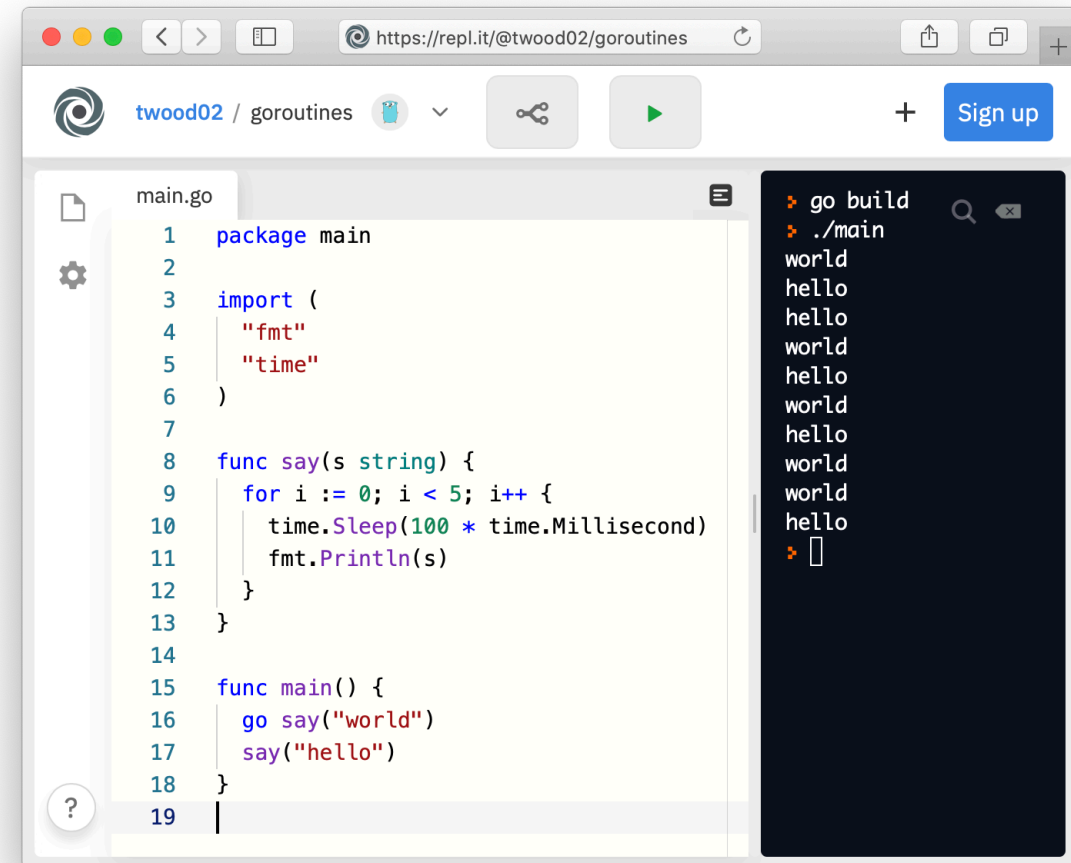
The right pane shows the output of the program after running the command `go build` and `./main`. The output is:

```
hello 0
hello 1
hello 2
hello 3
hello 4
```

<https://repl.it/@twood02/gofor>

PHASE 2: PARALLEL SUM

- Main thread still reads in file and makes array (see starter code)
- Use Goroutines to parallelize the addition
 - A **Goroutine** is a lightweight thread
 - **What does this mean with regards to concurrency and parallelism?**
- How will the main thread and goroutines coordinate?
 - Need to pass numbers to be summed
 - Need to get back the result
 - Hint: learn about **Go Channels!**



The screenshot shows a web-based Go Playground interface. The browser address bar displays <https://repl.it/@twood02/goroutines>. The editor shows a file named `main.go` with the following Go code:

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func say(s string) {
9     for i := 0; i < 5; i++ {
10         time.Sleep(100 * time.Millisecond)
11         fmt.Println(s)
12     }
13 }
14
15 func main() {
16     go say("world")
17     say("hello")
18 }
19
```

The output panel on the right shows the execution results:

```
go build
./main
world
hello
hello
world
hello
world
hello
world
hello
world
hello
```

<https://repl.it/@twood02/goroutines>

PHASE 3: HTTP+RPC

- Let's make a “real” distributed system! Two Go programs:
- HTTP Frontend
 - Accepts a client request specifying file to process
- RPC Backend
 - Receives a Remote Procedure Call from frontend to trigger the summation
 - Uses goroutines to parallelize like in prior phase

